

Trabajo Práctico Integrador 2026

Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

Materia: Programación Funcional

Grupo: 42

Lenguaje asignado (Fase 3): OCaml

Fecha de entrega: 16 de junio de 2026

Integrantes

#	Nombre y Apellido	Usuario de GitHub
1	(completar)	(completar)
2	(completar)	(completar)
3	(completar)	(completar)
4	(completar)	(completar)

1. Introducción y contexto

Las ciudades modernas requieren sistemas de tráfico inteligentes para optimizar el flujo vehicular y garantizar la seguridad vial. En este trabajo desarrollamos el núcleo lógico de un sistema embebido de control de semáforos para intersecciones críticas, implementado en Common Lisp aplicando estrictamente el paradigma funcional: funciones puras, inmutabilidad y composición.

El trabajo se organiza en tres fases:

- * Fase 1: lógica de control (Requerimientos 1 a 6) en Common Lisp.
- * Fase 2: integración del ecosistema con el gestor de paquetes Quicklisp (librería local-time): la auditoría (Req 3) pasa a mostrar la fecha y hora en formato legible en lugar del epoch Unix.
- * Fase 3: reimplementación de transicion y timer en OCaml y análisis comparativo entre paradigmas.

2. Fase 1 - Diseño funcional adoptado

2.1 Principios aplicados

Todo el núcleo (lisp/core.lisp) respeta las Restricciones de Diseño impuestas por la cátedra:

1. Inmutabilidad absoluta. No usamos defparameter/defvar para guardar estados cambiantes, ni operadores destructivos (setq, setf). El estado del tráfico (el color) y el tiempo (el timestamp Unix) fluyen únicamente como argumentos de las funciones. La configuración de tiempos se modela como una lista de asociación (alist) que se construye fresca en cada llamada a tiempos-por-defecto; nadie comparte ni puede mutar ese estado.
2. Cero bucles imperativos. No usamos loop, dolist, dotimes ni while.

Toda iteración se resuelve con:

- * Recursividad de cola (color-en-posicion), optimizada por SBCL para no consumir pila.

- * Funciones de orden superior (mapcar, reduce) en duracion-ciclo y distribucion-horaria.

3. Composición. Funciones pequeñas y puras se combinan para resolver problemas mayores: timer se apoya en duracion-ciclo y color-en-posicion; ciclos-por-tiempo y distribucion-horaria reutilizan duracion-ciclo; auditoria-quicklisp reutiliza timer para detectar el cambio de color.

2.2 Modelo de datos

- * Estados del semáforo: símbolos en-rojo, en-amarillo, en-verde.

- * Colores destino: símbolos rojo, amarillo, verde.

- * Configuración de tiempos: alist ((:rojo . 90) (:amarillo . 6) (:verde . 120)), construida fresca por la función pura tiempos-por-defecto (no es una variable global mutable). Al recibirse siempre por argumento, la fuente de los tiempos podría intercambiarse sin tocar el resto del código.

- * Secuencia temporal real: rojo -> verde -> amarillo -> (rojo). Es la base tanto del temporizador como de la tabla de transiciones válidas.

2.3 Requerimientos implementados

Req	Función(es)	Qué resuelve
1	transicion	Estado siguiente + acción "cambiar-a-<color>"; accion-por-defecto si la transición no es válida.
2	timer, color-en-posicion	Color activo en un timestamp Unix dado (ciclo de 216 s).
3	auditoria-quicklisp	Logging forense del cambio de estado: deriva los colores con timer y muestra la fecha legible con local-time.
4	duracion-ciclo, recomendacion-ciclo	Duración total del ciclo y recomendación según la regla psicológica (35-150 s).
5	ciclos-por-tiempo	Cantidad de ciclos completos en N minutos.
6	distribucion-horaria	Porcentaje de tiempo de cada color en 1 hora.
7	bloque de ejemplos	Casos normal / alternativo / error, listos para copiar y pegar.

Interpretaciones de ambigüedades del enunciado (documentadas para transparencia):

- * Req 1 - salida de transicion. El enunciado da como ejemplo (transicion 'en-rojo 'verde) -> ('en-rojo "cambiar-a-verde"): la lista contiene el estado actual y la acción (un literal string). Respetamos esa forma exacta. Las transiciones válidas siguen el ciclo real: en-rojo->verde, en-verde->amarillo, en-amarillo->rojo.

- * Req 4 - entrada de duracion-ciclo. El texto pide "calcular la duración que tendrá cada ciclo con las reglas de negocio actuales". Interpretamos que la entrada es la configuración de tiempos (no un único número suelto) y la salida, el total del ciclo (90+6+120 = 216 s). La evaluación psicológica del rango 35-150 s se delega en recomendacion-ciclo.

- * Req 6 - "porcentaje en 1 hora". Como el patrón es periódico, el porcentaje de cada color es la proporción dentro del ciclo, independiente de la hora: rojo ~ 41,67 %, amarillo ~ 2,78 %, verde ~ 55,56 %.

2.4 Clasificación taxonómica de las funciones

Cada función lleva en el código su encabezado de clasificación. Resumen:

Función	Naturaleza	Estrategia de control	Impacto en memoria
tiempos-por-defecto	Pura	Constructora simple	No destructiva
tiempo-de	Pura	Consulta/accesor (assoc)	No destructiva
secuencia-temporal	Pura	Constructora simple	No destructiva
duracion-ciclo	Pura	Orden superior (mapcar+reduce)	No destructiva
transiciones-validas	Pura	Constructora simple	No destructiva
transicion	Pura	Predicado + condicional	No destructiva
color-en-posicion	Pura	Recursiva de cola	No destructiva
timer	Pura	Composición	No destructiva
auditoria-quicklisp	Impura (imprime en pantalla)	Selección por condición (if) + interop	
local-time	No destructiva		
recomendacion-ciclo	Pura	Predicado (cond)	No destructiva
ciclos-por-tiempo	Pura	Composición (floor)	No destructiva
distribucion-horaria	Pura	Orden superior (mapcar)	No destructiva

3. Fase 2 - Autonomía y ecosistema (Quicklisp)

3.1 Quicklisp

Quicklisp es el gestor de paquetes de facto de Common Lisp. Se instala cargando quicklisp.lisp y ejecutando (quicklisp-quickstart:install); luego, cada librería se carga con (ql:quickload "nombre"). En core.lisp integramos local-time así:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (ql:quickload "local-time"))
```

El eval-when garantiza que la librería se cargue tanto al interpretar/cargar (load) como al compilar (compile-file) el archivo, de modo que los símbolos cualificados local-time:... ya existan cuando el reader lee la función de auditoría. Se asume Quicklisp instalado, que es precisamente el objetivo de la Fase 2 (investigar e integrar el gestor de paquetes).

3.2 Librería seleccionada y justificación

La consigna pide integrar una (1) de las dos librerías propuestas. Integramos local-time.

local-time - auditoría legible para humanos (Req 3).

La versión base del Requerimiento 3 imprime el instante en formato Unix (epoch), un entero como 1747405800 que es ilegible para un analista forense. Con local-time la auditoría muestra la fecha y hora en formato humano AAAA-MM-DD HH:MM:SS. La función auditoria-quicklisp recibe un único timestamp, deriva con timer el color del segundo anterior y el actual, y -si hubo un cambio- lo registra con la fecha ya formateada:

```
(defun auditoria-quicklisp (timestamp)
```

```
(let ((color-anterior (timer (- timestamp 1)))
      (color-actual (timer timestamp)))
      (if (not (equal color-anterior color-actual))
          (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
                  (local-time:format-timestring nil
                    (local-time:unix-to-timestamp timestamp)
                    :format '(:year 4) "-" (:month 2) "-" (:day 2)
                          " " (:hour 2) ":" (:min 2) ":" (:sec 2))
                    :timezone local-time:+utc-zone+)
                  color-anterior
                  color-actual))))
```

unix-to-timestamp convierte el epoch a un timestamp de local-time, y format-timestring lo compone con una lista de directivas. El formato se fija en UTC (+utc-zone+) para que la salida sea determinista y no dependa de la zona horaria de la máquina (ver bitácora, Bug 5).

Justificación. Frente a la alternativa (cl-json, que externaliza los tiempos a un .json), local-time ataca un problema real de calidad del dato de auditoría: la reconstrucción histórica de estados (el propósito explícito del Req 3) exige fechas legibles, no enteros epoch. Además, el manejo correcto de fechas, time zones y formateo es notoriamente propenso a errores si se hace a mano, por lo que apoyarse en una librería estándar y probada es la decisión de ingeniería más sólida. La integración es mínimamente invasiva: sólo cambia el borde de presentación del Req 3; el resto del núcleo (Req 1, 2, 4, 5, 6) permanece intacto y agnóstico de la librería.

3.3 Pureza vs. impureza en la integración

auditoria-quicklisp es impura: su propósito es el efecto de escribir en la terminal (logging). Aun así, todo el cálculo que la alimenta es puro: timer (que deriva los colores) y local-time:format-timestring / unix-to-timestamp (que arman la fecha en UTC) no tienen efectos secundarios ni mutan estructuras (es no destructiva). Se respeta así el patrón funcional clásico de "núcleo puro, cáscara impura": el efecto queda aislado en un único borde (el format), mientras el resto del sistema permanece puro.

4. Bitácora de Depuración

- | Se documentan de forma cruda los errores conceptuales surgidos en el desarrollo.
- | Cada bug incluye un espacio para la captura de pantalla del REPL y el
- | texto exacto que debe verse en ella.

Bug 1 - La lista citada '(...) no evalúa las variables

Síntoma. La primera versión de transicion devolvía el símbolo literal COLOR-ACTUAL en vez de su valor:

```
;; INCORRECTO
(defun transicion (color-actual cambiar-a)
  '(color-actual "cambiar-a-verde"))
```

```
(transicion 'en-rojo 'verde)
=> (COLOR-ACTUAL "cambiar-a-verde") ; <- esperábamos (EN-ROJO ...)
```

Causa. El quote '(...) suprime la evaluación de todo lo que contiene, incluidos los nombres de variable: Lisp devuelve el símbolo color-actual, no el argumento.

Solución. Construir la lista con list, que sí evalúa sus argumentos:

```
(list color-actual (format nil "cambiar-a-~(~a~)" cambiar-a))
```

[CAPTURA 1: REPL mostrando primero (COLOR-ACTUAL ...) y luego (EN-ROJO "cambiar-a-verde")]

Bug 2 - defconstant con listas falla al recargar el archivo

Síntoma. Habíamos definido la tabla de transiciones como constante global:

```
(defconstant +transiciones+ '((en-rojo . verde) ...))
```

Al recargar core.lisp por segunda vez SBCL abortaba con:

```
The constant +TRANSICIONES+ is being redefined (from ((EN-ROJO . VERDE) ...)
to ((EN-ROJO . VERDE) ...))
```

Causa. defconstant exige que el valor nuevo sea eql al anterior. Dos listas con el mismo contenido no son eql (son objetos distintos en memoria), así que cada recarga se interpreta como una redefinición ilegal.

Solución. Reemplazamos las "constantes" por funciones puras de cero argumentos (transiciones-validas, tiempos-por-defecto, secuencia-temporal) que devuelven una lista fresca. Esto además refuerza la inmutabilidad: no hay estado global, sólo funciones.

[CAPTURA 2: REPL con el error "constant ... is being redefined"]

Bug 3 - Desborde de pila por recursión mal planteada

Síntoma. Una versión temprana de color-en-posicion no reducía la posición ni acotaba con mod, y entraba en recursión infinita:

```
Control stack exhausted (no more space for function call frames).
```

Causa. El caso base (posicion < duracion) nunca se alcanzaba porque la llamada recursiva no decrementaba posicion, y timer pasaba el timestamp completo (millones) sin (mod timestamp ciclo). La pila se agotó.

Solución. (a) timer normaliza con (mod timestamp (duracion-ciclo ...)); (b) color-en-posicion resta la duración en cada paso -(- posicion duracion)- de forma tail-recursive, garantizando que el caso base se alcance.

[CAPTURA 3: REPL con "Control stack exhausted" y luego la versión corregida devolviendo :VERDE]

Bug 4 - Los símbolos se imprimían en MAYÚSCULAS

Síntoma. La acción salía como "cambiar-a-VERDE" en lugar de "cambiar-a-verde".

Causa. El reader de Common Lisp convierte los símbolos a mayúsculas por defecto ('verde se internaliza como VERDE), y ~A los imprime tal cual.

Solución. Usar la directiva de case de format: ~(~a~), que pasa a minúsculas el texto generado: (format nil "cambiar-a-~(~a~)" 'verde) -> "cambiar-a-verde".

[CAPTURA 4: REPL mostrando "cambiar-a-VERDE" y luego "cambiar-a-verde"]

Bug 5 - La auditoría mostraba una hora corrida (zona horaria) (Fase 2)

Síntoma. Al integrar local-time, la auditoría del epoch 1747405800 imprimía una hora distinta en cada máquina (corrida varias horas) y no coincidía con la esperada en UTC:

```
;; INCORRECTO (sin fijar zona): toma la zona horaria local de la maquina
(local-time:format-timestring nil (local-time:unix-to-timestamp 1747405800)
                               :format '(:year 4) "-" (:month 2) "-" (:day 2)))
=> "2025-05-16 11:30:00" ; <- corrido -3 h respecto de UTC (zona AR)
```

Causa. format-timestring, si no se le pasa :timezone, usa local-time:default-timezone, que depende del entorno. La salida dejaba de ser determinista y los ejemplos del Req 7 no eran reproducibles.

Solución. Formatear siempre en UTC, pasando explícitamente :timezone local-time:+utc-zone+ dentro de auditoria-quicklisp. Así la fecha se imprime igual en cualquier máquina.

[CAPTURA 5: REPL mostrando primero la hora corrida y luego, con +utc-zone+, la hora en UTC]

Bug 6 - El format no imprimía la fecha ni saltaba de línea (Fase 2)

Síntoma. La primera versión de auditoria-quicklisp mostraba mal la línea: aparecía una A literal, la fecha caía en el lugar equivocado, faltaba un color y la línea terminaba con un % suelto en vez de saltar de renglón:

```
;; INCORRECTO
(format t "Tiempo A: la luz ha cambiado de ~A a ~A%"
        fecha color-anterior color-actual)
;; => "Tiempo A: la luz ha cambiado de 2025-05-16 14:30:00 a ROJO%"
```

Causa. Dos errores de directivas de format: (1) se escribió A literal en vez de la directiva ~A, por lo que la fecha no quedaba tras "Tiempo " y los ~A se "corrían" (la fecha ocupaba el lugar del color y color-actual no se imprimía); (2) % es un carácter literal -la directiva de nueva línea en format es ~%.

Solución. Usar ~A para los tres argumentos (fecha, color-anterior, color-actual) y ~% para el salto de línea:

```
(format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
        fecha color-anterior color-actual)
```

[CAPTURA 6: REPL mostrando primero la línea con "Tiempo A:" y el "%" literal, y luego la

línea correcta]

5. Fase 3 - Estudio Comparativo: OCaml

5.1 Presentación del lenguaje

OCaml (Objective Caml, 1996, INRIA - Francia) es un lenguaje funcional con tipado estático fuerte e inferencia de tipos (sistema Hindley-Milner), que también admite estilo imperativo y orientado a objetos (la "O"). Sus rasgos distintivos son los tipos algebraicos (variantes), el pattern matching exhaustivo, la inmutabilidad por defecto y un compilador a código nativo muy veloz. Su lema informal es "si compila, probablemente funciona": el verificador de tipos elimina en compilación clases enteras de errores.

Industrias y áreas donde se usa:

- * Finanzas / trading de alta frecuencia: es su nicho más famoso por la seguridad de tipos y el rendimiento.
- * Compiladores y tooling de lenguajes: análisis estático, type checkers.
- * Métodos formales y verificación: asistentes de prueba y verificadores.
- * Blockchain: contratos e infraestructura.
- * Sistemas e infraestructura.

Empresas y proyectos conocidos:

- * Jane Street - firma financiera que programa casi todo su trading en OCaml; principal patrocinadora del ecosistema.
- * Meta (Facebook) - Flow (type checker de JavaScript), Hack y Infer (análisis estático) están escritos en OCaml; Reason es una sintaxis alternativa para OCaml creada allí.
- * Docker - su stack de red original (MirageOS / unikernels) usaba OCaml.
- * Tezos - la blockchain está implementada en OCaml.
- * Citrix (XenServer) - el toolstack de Xen está escrito en OCaml.
- * Coq / Rocq - el asistente de pruebas formales está implementado en OCaml.
- * El primer compilador de Rust se escribió en OCaml antes de hacerse self-hosted.

5.2 Reimplementación de transicion y timer

El código completo está en comparativa/solucion.ml. Núcleo:

```
type estado = En_rojo | En_amarillo | En_verde
type color  = Rojo | Amarillo | Verde
```

```
let transicion actual cambiar_a =
  match actual, cambiar_a with
  | En_rojo, Verde -> (En_rojo, "cambiar-a-verde")
  | En_verde, Amarillo -> (En_verde, "cambiar-a-amarillo")
  | En_amarillo, Rojo -> (En_amarillo, "cambiar-a-rojo")
  | _, _ -> (actual, "accion-por-defecto")
```

```
let tiempo_rojo = 90 and tiempo_verde = 120 and tiempo_amarillo = 6
```

```

let duracion_ciclo = tiempo_rojo + tiempo_verde + tiempo_amarillo (* 216 *)

let timer timestamp =
  let posicion = ((timestamp mod duracion_ciclo) + duracion_ciclo) mod duracion_ciclo in
  if posicion < tiempo_rojo then Rojo
  else if posicion < tiempo_rojo + tiempo_verde then Verde
  else Amarillo

```

5.3 Pregunta 1 - Inferencia de tipos

| *Al igual que Haskell, OCaml usa tipos estáticos pero cuenta con Inferencia de Tipos. ¿Tuvieron que declarar explícitamente de qué tipo eran las funciones o el compilador lo dedujo solo? Muestren cómo lo interpreta el entorno.*

No declaramos ningún tipo a mano: el compilador los dedujo solo. Al pegar las definiciones en el toplevel (ocaml / utop), el entorno responde con la firma inferida (líneas val ...):

```

# type estado = En_rojo | En_amarillo | En_verde;;
type estado = En_rojo | En_amarillo | En_verde

# type color = Rojo | Amarillo | Verde;;
type color = Rojo | Amarillo | Verde

# let transicion actual cambiar_a =
  match actual, cambiar_a with
  | En_rojo, Verde -> (En_rojo, "cambiar-a-verde")
  | En_verde, Amarillo -> (En_verde, "cambiar-a-amarillo")
  | En_amarillo, Rojo -> (En_amarillo, "cambiar-a-rojo")
  | _, _ -> (actual, "accion-por-defecto");;
val transicion : estado -> color -> estado * string = <fun>

# let timer timestamp =
  let posicion = ((timestamp mod 216) + 216) mod 216 in
  if posicion < 90 then Rojo
  else if posicion < 210 then Verde
  else Amarillo;;
val timer : int -> color = <fun>

```

¿Cómo lo deduce? El motor Hindley-Milner razona a partir del uso:

* En transicion, hacer match del primer argumento contra el constructor En_rojo obliga a que actual : estado; contra Verde, a que cambiar_a : color; los -> devuelven una tupla (estado, string), de donde infiere el retorno estado * string.
 * En timer, el operador mod exige int, y las ramas devuelven constructores de color; de ahí int -> color.

A diferencia de Common Lisp (tipado dinámico: no hay verificación previa y un (transicion "rojo" 'verde) falla recién en ejecución), OCaml rechaza en compilación cualquier uso con tipos incompatibles. Las anotaciones son opcionales: podríamos escribir

```
let transicion (actual : estado) (cambiar_a : color) : estado * string = ... como
```


documentación o para restringir, pero no son necesarias.

5.4 Pregunta 2 - Orientación a expresiones y match...with

| *OCaml es un lenguaje "orientado a expresiones". ¿Cómo cambia esto la estructura
| de control de flujos (como el uso de match...with) comparado con los
| condicionales de Lisp?*

En OCaml todo es una expresión que produce un valor: no existen "sentencias".
if ... then ... else ... y match ... with devuelven un valor, igual que en Lisp
cond/case (Common Lisp también es, en este sentido, orientado a expresiones).
La diferencia real no es "expresión vs. sentencia", sino tres ventajas del
match...with de OCaml frente a los condicionales de Lisp:

1. Descomposición estructural (pattern matching). En transición hacemos
match sobre la tupla (actual, cambiar_a) y desarmamos los datos en un
solo paso. En Lisp habría que encadenar tests booleanos:

```
;; Lisp: cond con predicados explícitos
(cond ((and (eq color-actual 'en-rojo) (eq cambiar-a 'verde))
      (list color-actual "cambiar-a-verde"))
      ;; ... más cláusulas ...
      (t (list color-actual 'accion-por-defecto)))

(* OCaml: el patrón describe la forma del dato *)
match actual, cambiar_a with
| En_rojo, Verde -> (En_rojo, "cambiar-a-verde")
| _, _           -> (actual, "accion-por-defecto")
```

2. Exhaustividad verificada en compilación. Si olvidamos un caso, el
compilador avisa: "this pattern-matching is not exhaustive". cond/case
de Lisp no ofrecen esa garantía: un caso faltante pasa silenciosamente al t/
otherwise (o devuelve nil) y el bug aparece en ejecución.

3. Seguridad de tipos. Como estado y color son tipos cerrados, sólo se
pueden matchear sus constructores válidos; un "azul" inexistente ni siquiera
compila. En Lisp, case compara por eql contra símbolos arbitrarios sin
chequeo previo.

El comodín _ cumple el papel del t de cond o del otherwise de case: la
cláusula por defecto. En síntesis, match...with no sólo elige una rama: describe
la forma de los datos y obliga a cubrir todos los casos, trasladando errores de
tiempo de ejecución (Lisp) a tiempo de compilación (OCaml).

5.5 Conclusión del grupo

Estudiar OCaml después de Common Lisp fue revelador. Veníamos de un mundo donde el
error aparece al ejecutar; en OCaml muchos de esos errores se vuelven
imposibles: el tipo estado no admite valores inválidos y el compilador exige
cubrir todos los casos del match. Eso obliga a pensar el dominio antes de
escribir la lógica, lo que resulta incómodo al principio y muy tranquilizador
después. La inferencia de tipos nos dio lo mejor de dos mundos: la seguridad del
tipado estático sin la verbosidad de declarar todo. Comparado con los paréntesis
de Lisp, el pattern matching sobre tuplas nos pareció notablemente más

declarativo y legible. La mayor fricción fue acostumbrarnos a que "todo es una expresión" (por ejemplo, que un if sin else deba devolver unit) y a leer los mensajes de tipo del compilador, que al principio intimidan pero terminan siendo la mejor documentación.

6. Bibliografía

- * Quicklisp - gestor de paquetes de Common Lisp. <https://www.quicklisp.org/>
- * local-time - manejo de fechas, horas y zonas horarias en CL.
<https://common-lisp.net/project/local-time/> y <https://github.com/dlowe-net/local-time>
- * Seibel, P. Practical Common Lisp. <https://gigamonkeys.com/book/>
- * SBCL - Steel Bank Common Lisp. <http://www.sbcl.org/>
- * OCaml - sitio oficial y manual. <https://ocaml.org/> y <https://ocaml.org/manual/>
- * Real World OCaml (Minsky, Madhavapeddy, Hickey).
<https://dev.realworldocaml.org/>
- * OCaml Playground (entorno en línea). <https://ocaml.org/play>
- * Jane Street Tech Blog (uso industrial de OCaml).
<https://blog.janestreet.com/>